



空间感知与计算实验室
spAtial Sensing & Computing Lab

CSC4301 Artificial Intelligence

Dr. Saifullahi Aminu Bello



Introduction to Artificial Intelligence



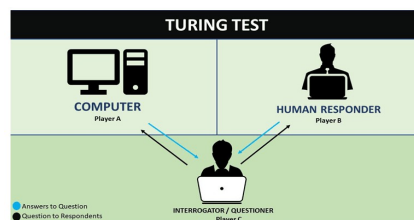
Definition of AI

- A branch of science that deals with creating a machine capable of reasoning, learning, and performing task that require human intelligence.
 - E.g visual perception, speech recognition, decision-making, and language translation, etc
- Integrates areas such as computer science, neuroscience, philosophy, psychology.



Turing Test

- Proposed in 1950 by Alan Turing
- a way to check if a machine can act "intelligent" enough to fool a human into thinking it's also human.

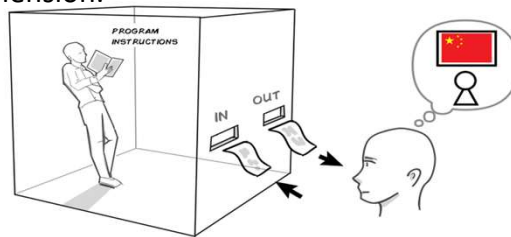


- Does passing Turing test equates to Intelligence? Remains a subject of debate.



Chinese Room

- proposed by philosopher John Searle in 1980
- It challenges the idea that AI can truly understand language or possess a mind like humans
- argues a computer following programmed rules to process language does not truly "understand" it but only manipulates symbols. Similar to a person who don't understand Chinese, but can follow rules to provide correct answer based on the input.
- It suggests that AI systems, no matter how advanced, might simulate understanding without genuine comprehension.



- It is still an ongoing debate.



History of AI

- The Birth of AI (1940s - 1950s)
 - Turing test was introduced in 1950 paper "Computing Machinery and Intelligence."
 - In 1956 the first conference on AI by 10 attendees was held in Dartmouth US

- Early AI Research (1956 - 1970s)
 - After the conference symbolic AI followed





History of AI

- **AI winter (1970s - 1980s):** AI winters were periods of reduced funding and interest in AI due to unmet expectations:
 - AI systems struggled with complex, real-world problems
 - Making governments to cuts funding
 - 1980s saw renewed interest with the development of expert systems, but high costs and limitations led to another decline



History of AI

- **Resurgence and Machine Learning (1990s - 2010s)**
 - Improved computing power
 - More data becomes available
 - Neural Nets and Backpropagation becomes effective
 - In 1997 Deep Blue beats the best chess player in the world
 - In 2000s speech recognition, Google search, self-driving cars start to become popular.



History of AI

- **The Deep Learning Revolution (2010s - Present)**
 - CNNs revolutionized image processing (e.g., AlexNet in 2012).
 - AI is now embedded in daily life, from virtual assistants to autonomous vehicles



Types of AI

- AI can be categorized based on two major criteria:
 - 1) Based on Capabilities**
 1. Narrow AI
 2. General AI
 3. Super AI
 - 2) Based on Functionalities**
 - 1) Reactive AI
 - 2) Limited Memory AI
 - 3) Theory of Mind
 - 4) Self-Aware AI



Types of AI based on capabilities

- **Narrow AI (Weak AI)**
 - They are designed and trained for specific tasks. And cannot perform beyond their specific functions
 - Examples include virtual assistant such as Siri, Language Models such as ChatGPT, Recommendation systems like the one used in Netflix.
- **General AI (Strong AI):**
 - Are AI systems that possess the ability to understand, learn, and apply intelligence across a wide range of tasks at a level comparable to human cognition.
 - They can transfer knowledge from one domain to another and can improve themselves autonomously
 - General AI remains theoretical
- **Super AI**
 - Are AI systems that surpasses human intelligence in all aspects, including creativity, problem-solving, and decision-making.
 - They are capable of self-awareness, consciousness, and autonomous self-improvement
 - purely hypothetical and re-mains a subject of debate among researchers and philosophers.



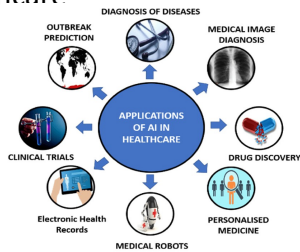
Types of AI based on functionalities

1. **Reactive AI**
 - Are the most Basic AI systems that do not store past experiences or learn from past experiences. They operate solely on predefined rules and immediate input.
 - Examples are IBM's Deep Blue and Google's AlphaGo
2. **Limited Memory AI**
 - These are the AI systems of today. They can retain past experiences for a short duration and use them to make better decisions. Deep learning is what makes limited memory AI possible.
 - Examples include Autonomous cars, Face recognition systems, etc.
3. **Theory of Mind AI**
 - These type of AI would be able to understand human emotions, beliefs, intentions, and thoughts. They are sill in early stages of development.
 - Example is the robot named Sophie, a citizen of Saudi Arabia.
4. **Self-aware AI**
 - They are the most advanced AI that are not only aware of emotions and mental states of others, but also their own.
 - However, No existing AI system has reached this level. It remains a subject of speculation and debate.



Applications of AI today

- Healthcare



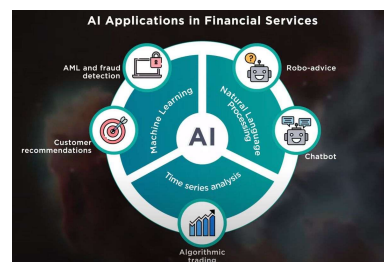
- Manufacturing



- Autonomous Driving



- Finance



空间感知与计算实验室 13
spAtial Sensing & Computing Lab



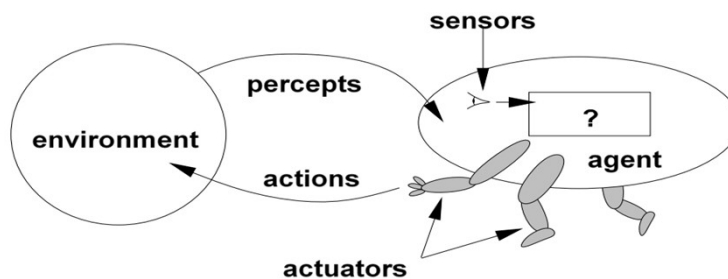
Intelligent Agent

ASC 空间感知与计算实验室 14
spAtial Sensing & Computing Lab



Introduction

- An **intelligent agent** is an entity that perceives its **environment** through **sensors** and acts upon it using **actuators** to achieve its goals.



- **Agent function** maps percepts to actions $f: P^* \rightarrow A$
- **Agent Program** is the actual implementation of the agent function



Rationality

- **Rationality** refers to an agent's ability to make the best possible decision based on available information, goals, and its knowledge of the environment.
- **Omniscience** refers to an agent's ability to have **complete, perfect knowledge** about its environment, all possible outcomes, and the best possible actions at all times. (not achievable in real-world)



Rational Agent

- An ideal **rational agent** is the one, which is capable of doing expected actions to maximize its performance measure.
- Rationality of an agent depends on the following:
 - 1. The performance measures, which determine the degree of success.
 - 2. Agent's Percept Sequence till now.
 - 3. The agent's prior knowledge about the environment.
 - 4. The actions that the agent can carry out.



Task Environment

- Task environments are essentially the “problem” for which the rational agent is a “solution” of.
- It includes everything the agent interacts with to perform its tasks **rationally**.
- Grouped as PEAS
 - Performance measure - How success is measured.
 - Environment - The surroundings where the agent operates.
 - Actuators - The mechanisms the agent uses to take action
 - Sensors - The tools used to perceive the environment.



PEAS Example

	Self Driving taxi	Google search engine	Smart Camera
Performance Measure	safety, destination, profits	Relevance and speed of search results	Detecting movement accurately, minimizing false alarms
Environment	Roads, traffic, pedestrians, weather conditions	Web pages, user queries, ranking algorithms	House surroundings (indoor/outdoor)
Actuators	Steering, accelerator, brakes, signals, Horn	Displaying ranked search results	Camera recording, sending alerts
Sensors	Cameras, LIDAR, GPS, speedometer	User input (keywords), click patterns, webpage data	Motion detector, microphone for sound detection



Agent Environment

- An **Environment** is the surrounding in which an agent operates.
- Properties of the environment
 - fully observable/partially observable
 - Deterministic/Stochastic/strategic
 - Episodic/sequential
 - Static/dynamic/semi-dynamic
 - Discreet/continues
 - Single agent/multiagent



Fully observable/Partially observable

- **Fully observable:** An agent's sensors give it access to the complete state of the environment at each point in time. Eg Chess Playing AI
- **Partially observable:** because of noisy and inaccurate sensors or because parts of the state are simply missing from the sensor data. Eg Self Driving car



Deterministic/Stochastic/strategic

- **Deterministic:** The next state of the environment is completely determined by the current state and the action executed by the agent. Eg Solitaire.
- **Stochastic:** Otherwise, is stochastic. E.g self-driving taxi (there may be an engine failure, tire blowout, actions of other drivers, traffic behaviour)
- **Strategic:** If the environment is deterministic except for the actions of other agents. Eg Chess



Episodic/sequential

- **Episodic:** The agent's experience is divided into atomic "episodes".
 - each episode consists of the agent perceiving and then performing a single action
 - the choice of action in each episode depends only on the episode itself.
 - Eg Face recognition, Spam email classifier
- **Sequential:** the current decision could affect all future decisions. Eg Chess, self-driving taxi
- Episodic are simpler than sequential, agent need not to think



Static/dynamic/semi-dynamic

- **Static:** The environment is unchanged while an agent is making a decision. Eg Chess
- **Dynamic:** The environment while the agent is deciding or taking action. Eg self driving taxi.
- **Semi-Dynamic:** Environment does not change independently, but time constraints the performance of the agent. Eg Chess with a clock.



Discreet/continues

- **Discreet:** A limited number of distinct, clearly defined percepts and actions. Eg Chess
- **Continues:** percepts and actions are infinite or uncountable. Eg self-driving cars (Speed, steering, positions are continues variables)



Single agent/multiagent

- **Single agent:** An agent operating by itself in an environment. E.g. puzzle solving agent.
- **Multi agent:** Multiple agents exist in the environment, and they can either **cooperate** or **compete** with each other. E.g. Chess (two agents competing), Online Multiplayer Games (each team cooperate to defeat their opponents).



Example

	Chess	Solitaire	Self driving car
fully observable	Yes	Yes	No
Deterministic	Strategic	Yes	No
Episodic	No	No	No
Static	Yes	Yes	No
Discreet	Yes	Yes	No
Single agent	No	Yes	No



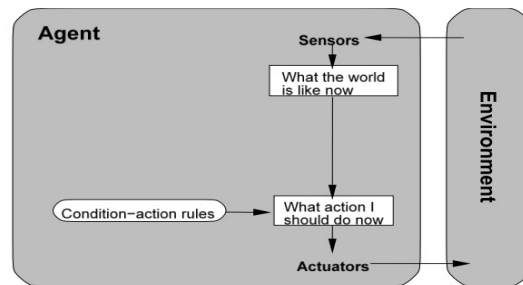
Agent Types

- Agent types are determine by the agent program.
- The type of agents defines what actions to be taken based on the current percepts. They include:
 - simple reflex agents
 - Model-based reflex agents
 - goal-based agents
 - utility-based agents



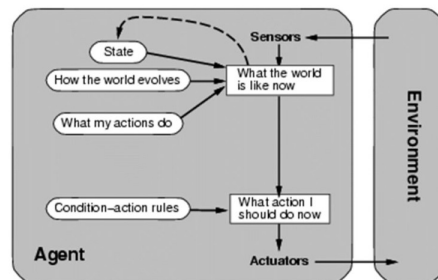
simple reflex agents

- The simplest kind of agent. These agents select actions on the basis of the current percept, ignoring the rest of the percept history.
 - Condition-action rules
 - No Past Memory
 - Limited Intelligence
 - Need Fully observable env.
- Example
 - Thermostat
 - Automatic Street Light
 - Automatic water dispenser



Model-based Reflex Agents

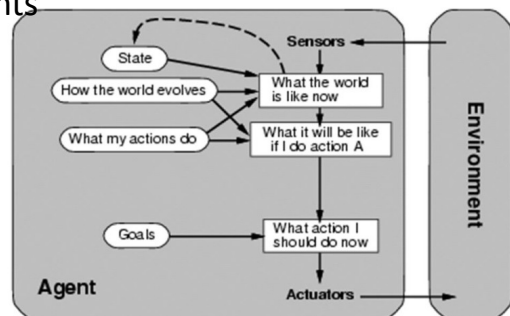
- improves on a simple reflex agent by **incorporating a model of the world**.
- It **maintains internal states** based on its perception of the environment, enabling it to **handle partial or dynamic environments** better than simple reflex agents.
 - Internal state (memory of the past)
 - Works on condition-action rules
 - Handles dynamic environment
- Examples
 - Self driving car stopping at the red light.
 - When the traffic light is green, based on its distance, speed May decide to speed on, or slow down and stop.
 - Once stopped, it update its internal state to stop.





Goal-based agents

- More advanced.
- It doesn't just react to its environment based on **what it sees** or **remembers**; it also works towards achieving **specific goals**.
 - Consider the future goal
 - More flexible than model-based agents
- Example: Self driving taxi
 - May stop at red light
 - But may change the route if it anticipates heavy traffic ahead

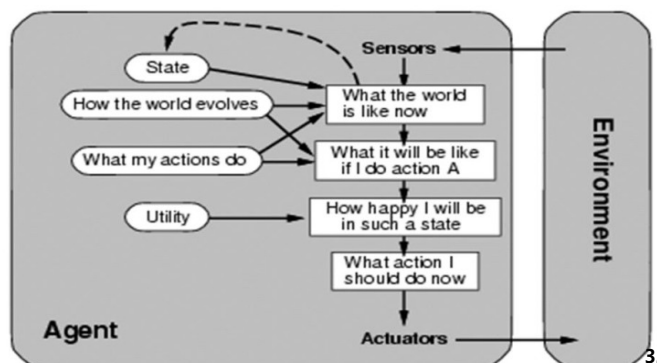


31



Utility-based agents

- They may have multiple ways to achieve a goal.
- These agents go a step further by choosing actions that **maximize their utility**.
- **Utility** is the benefit they receive from performing certain actions. (internalization of performance measure)
 - Have multiple options
 - Always choose options to maximize their utility (save time, save fuel, etc)



32



Solving Problems by Searching



Previous class

- Simple reflex and model based reflex agents have direct mapping from state to action.
- Goal based models don't have direct mapping from state to action. They consider future actions and desirability of their outcome.



Problem solving agent

- A Problem-solving agent is an agent that searches for and executes a sequence of actions to reach a defined goal efficiently.
- A kind of goal-based agent is the **problem-solving agent**.
- Is widely used in robotics, gaming, AI planning, and decision-making systems.

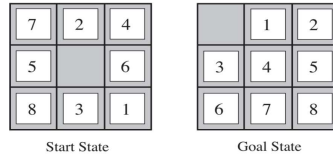


Problem and Solution

- A **problem** can be defined formally by five components:
 1. **Initial State:** The state that the agent starts in.
 2. **Actions:** A description of the possible actions available to the agent.
 3. **Transition Model:** A description of what each action does.
 4. **Goal Test:** determines whether a given state is a goal state.
 5. **Path Cost:** A function that assigns a numeric cost to each path.
- A **solution** is a sequence of actions that leads from the initial state to a goal state.



Examples 8-Puzzle



- **Initial state:** Any state can be initial state
- **Actions:** Left, right, up, down
- **Transition model:** Outcome of a state action pair. Eg move 5 right, the resultant model has 5 and blank interchanged
- **Goal Test:** Check if state matches goal state
- **Path Cost:** Each step costs 1, so path cost is the total number of steps



Examples GPS navigation system

- **Initial State:** The current location of the vehicle.
- **Actions:** Possible movements, such as moving forward, turning left, turning right, or stopping.
- **Transition Model:** Defines how actions change the vehicle's state; for example, if the car moves forward on a road, its coordinates update accordingly.
- **Goal State:** The destination where the vehicle needs to arrive.
- **Path Cost:** The total cost of a chosen path, which may depend on distance, time, fuel consumption, toll fees, or traffic conditions.



Searching for solution

- A **state space** is the **set of all possible states** that an AI agent can be in while solving a problem.
- **Searching** is the process of exploring a state space to find a sequence of actions that leads from an initial state to a goal state.
-
- There are **two** types of searching algorithms
 - 1. **Uninformed search**- No prior knowledge about the goal
 - 2. **Informed search**- Uses heuristics to guide the search.



Search Tree Concept

- A **search tree** represents all possible action sequences from the **initial state** to the **goal state**.
 - The **root node** is the **initial state**.
 - The **branches** represent **actions** taken.
 - The **nodes** represent **states** reached after each action.
 - The **goal state** is the target node where the solution is found.
- For each node, we have 4 components
 - State
 - Parent
 - Action
 - Path cost



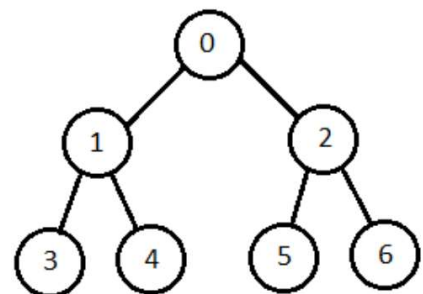
Algorithm evaluation

- **Completeness:** Is the algorithm guaranteed to find a solution when there is one?
- **Optimality:** Does the strategy find the optimal solution?
- **Time Complexity:** How long does it take to find a solution?
- **Space Complexity:** How much memory is needed to perform the search?



Uninformed Search - Breadth-first search (BFS)

- Explores all nodes at the current depth level before moving deeper.
- Uses a **queue (FIFO)** to store nodes.
- Advantages
 - Guarantees the shortest path in an unweighted graph.
 - Complete (always finds a solution if one exists).
- Disadvantages
 - High memory
 - Slow for deep search



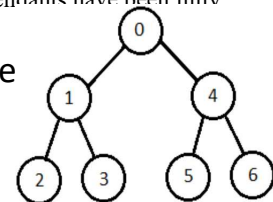


Uninformed Search - Depth-first search (DFS)

- Explores as far as possible along a branch before backtracking.
- Uses a **stack (LIFO)** for node storage.
- Advantages
 - Uses less memory than BFS.

(Once a node has been expanded, it can be removed from memory as soon as all its descendants have been fully explored.)

- Good for problems where solutions are deep in the tree
- Disadvantages
 - May get stuck in infinite loops
 - Does not guarantee the shortest path.

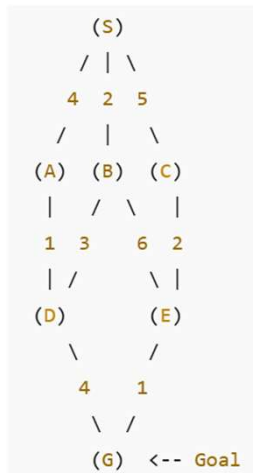


Uninformed Search - Uniform-cost search (UCS)

- Expands the least-cost node first, prioritizing paths with the lowest total cost.
- Uses a **priority queue** (based on path cost).
- Advantages
 - Guarantees optimal path
 - Works well with weighted graphs.
- Disadvantages
 - Can be slow if all edge costs are the same (behaves like BFS).
 - Requires high memory.



Uninformed Search - Uniform-cost search (UCS)



1. Start at **S** with cost 0
 - Queue: [(S, 0)]
2. Expand **S** → Add (A, 4), (B, 2), (C, 5) to the queue
 - Queue: [(B, 2), (A, 4), (C, 5)] (sorted by cost)
3. Expand **B** (lowest cost: 2) → Add (D, 3+2=5), (E, 6+2=8)
 - Queue: [(A, 4), (D, 5), (C, 5), (E, 8)] (sorted by cost)
4. Expand **A** (lowest cost: 4) → Add (D, 4+1=5)
 - Queue: [(D, 5), (D, 5), (C, 5), (E, 8)] (sorted by cost)
5. Expand **D** (lowest cost: 5) → Add (G, 5+4=9)
 - Queue: [(D, 5), (C, 5), (E, 8), (G, 9)] (sorted by cost)
6. Expand **D** again (another path, cost 5, already explored)
 - No change.
7. Expand **C** (cost 5) → Add (E, 5+2=7)
 - Queue: [(E, 7), (E, 8), (G, 9)] (sorted by cost)
8. Expand **E** (lowest cost: 7) → Add (G, 7+1=8)
 - Queue: [(E, 8), (G, 8), (G, 9)] (sorted by cost)
9. Expand **G** (lowest cost: 8) → Goal found!

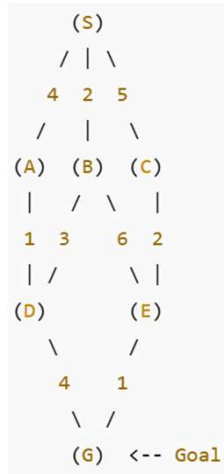


Uninformed Search - Bidirectional Search

- Runs two simultaneous searches:
 - From the start node
 - From the goal node (the goal is known in advance)
- The searches meet in the middle.
- Advantages
 - Much faster than unidirectional searches.
 - Reduces the number of nodes explored.
- Disadvantages
 - Requires two search trees and extra memory.
 - Hard to implement if goal states are not explicitly defined.



Uninformed Search - Bidirectional Search



1. Start Two Searches Simultaneously:
 - Forward search from $S: (S, 0)$
 - Backward search from $G: (G, 0)$
2. Expand $S \rightarrow$ Add $(A, 4), (B, 2), (C, 5)$
 - Forward Queue: $[(B, 2), (A, 4), (C, 5)]$ (sorted by cost)
3. Expand $G \rightarrow$ Add $(D, 4), (E, 1)$
 - Backward Queue: $[(E, 1), (D, 4)]$ (sorted by cost)
4. Expand B (lowest cost: 2) $\rightarrow$ Add $(D, 5), (E, 8)$
 - Forward Queue: $[(A, 4), (C, 5), (D, 5), (E, 8)]$
5. Expand E (lowest cost: 1, from the goal side) $\rightarrow$ Add $(C, 3), (B, 7)$
 - Backward Queue: $[(C, 3), (D, 4), (B, 7)]$
6. Intersection Found! C is in both forward and backward queues.
 - Forward path to $C: S \rightarrow C$ (cost 5)
 - Backward path to $C: G \leftarrow E \leftarrow C$ (cost 3)
 - Final path: $S \rightarrow C \rightarrow E \rightarrow G$ with cost 8

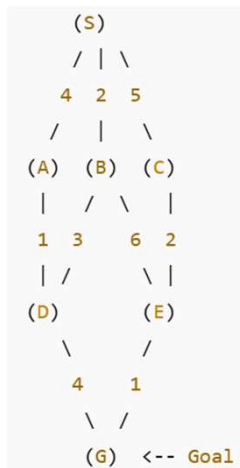


Uninformed Search - Iterative Deepening Depth-First Search (IDDFS)

- A combination of DFS and BFS:
 - Repeatedly runs DFS with increasing depth limits until the goal is found.
- **Advantages:**
 - Uses less memory than BFS.
 - Guarantees finding the shortest path.
- **Disadvantages:**
 - Re-explores nodes multiple times, making it inefficient in some cases.



Uninformed Search - Iterative Deepening Depth-First Search



- Depth = 0: $\{S\}$ ✗
- Depth = 1: $\{S \rightarrow A, B, C\}$ ✗
- Depth = 2: $\{S \rightarrow A \rightarrow D, S \rightarrow B \rightarrow D, E, S \rightarrow C \rightarrow E\}$ ✗
- Depth = 3: $\{S \rightarrow A \rightarrow D \rightarrow G\}$ ✓ Goal Found!

✓ Solution Path:

$S \rightarrow A \rightarrow D \rightarrow G$ (Total Cost: $4 + 1 + 4 = 9$)



Uninformed Search

Criterion	Breadth First	Depth First	Bidirectional	Uniform Cost	Iterative Deepening
Time	b^d	b^m	$b^{d/2}$	b^d	b^d
Space	b^d	b^m	$b^{d/2}$	b^d	b^d
Optimality	Y	N	Y	Y	Y
Completeness	Y	N	Y	Y	Y

- b is average the number of child in each node
- d is the depth
- m is the maximum depth of any node in DFS
- Y=Yes
- N=No



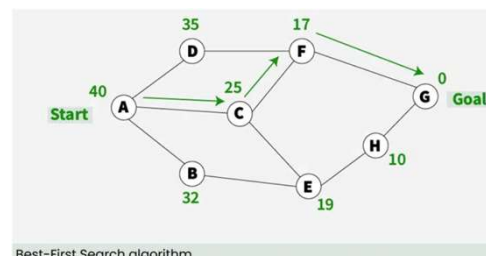
Informed Search (Heuristic Search)

- To solve large problems with large number of possible states
- problem-specific knowledge is added to increase the efficiency of search algorithms.
- **Heuristic** is an **estimated guide** used in prioritizing promising paths.



Informed Search -Greedy best-first search (GBFS)

- Greedy Best-First Search is an AI search algorithm that attempts to find the most promising path from a given starting point to a goal.
- Advantage
 - Simple and easy to implement
 - Fast
- Disadvantage
 - No guarantee to find optimal solution
 - It can get stuck in a loop



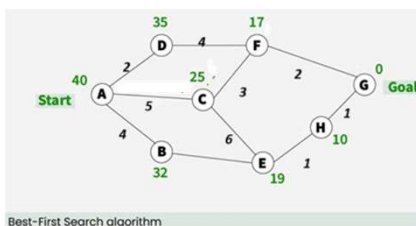


Informed Search -A* Search

- A* Search is one of the most widely used informed search algorithms.

- Combines uniform cost search and greedy best search

$$- f(n)=g(n)+h(n)$$



Green numbers are the heuristic value,
black are path cost

A->C (5+25) = 30 best

C->F (5+3+17) = 25 best compared to C->E (5+6+19)=30

F-> G (5+3+2+0)=10 Goal reached

A->C->F->G solution



Lecture 4 Fuzzy Logic Systems



Introduction

- In previous lecture, we covered **Problem-Solving Agents** and **Search Algorithms**.
- Search techniques work well for structured problems with well-defined states and solutions.
- However, real-world AI applications often involve **uncertainty and imprecision** (e.g., language, human behavior, sensor data).
- **Fuzzy Logic** helps AI handle vague and approximate reasoning where traditional search methods struggle.



Introduction

- **Fuzzy Logic** is a mathematical system for handling imprecise and uncertain information.
- Fuzzy logic, a concept introduced by Lotfi Zadeh in the 1960s
- It models human reasoning and decision-making.
- Unlike **classical logic** (True/False), **fuzzy logic** works with degrees of truth (0 to 1).

Classical Logic	Fuzzy Logic
0 or 1 (True/False)	Any value between 0 and 1
Strict boundaries	Flexible and gradual changes
Example: Is the temperature hot? (Yes/No)	Example: Temperature is 70% hot



Key Concepts in Fuzzy Logic

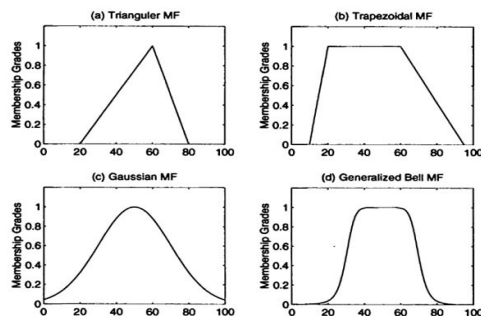
1. **Fuzzy Sets:** A fuzzy set is a set without a sharp boundary. The membership of elements in a fuzzy set is described by a membership function that assigns each element a degree of membership ranging from 0 to 1.
2. **Membership Functions:** These functions define how each point in the input space is mapped to a membership value (or degree of membership) between 0 and 1.

- Mathematical representation

$$\mu_A: X \rightarrow [0,1]$$

- Common membership functions include

- triangular, trapezoidal, and Gaussian functions.



ASC 空间感知与计算实验室 57
spAtial Sensing & Computing Lab



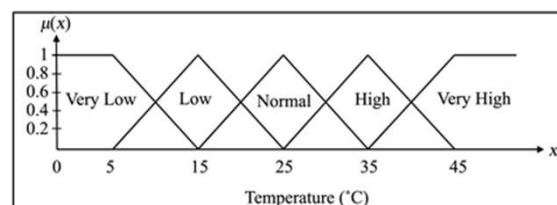
Key Concepts in Fuzzy Logic

1. **Fuzzy Sets:** A fuzzy set is a set without a sharp boundary. The membership of elements in a fuzzy set is described by a membership function that assigns each element a degree of membership ranging from 0 to 1.
2. **Membership Functions:** These functions define how each point in the input space is mapped to a membership value (or degree of membership) between 0 and 1.

- Mathematical representation

$$\mu_A: X \rightarrow [0,1]$$

- Example: Room Temperature



x axis represents the universe of discourse
y axis represents the membership value in the interval [0,1]

ASC 空间感知与计算实验室 58
spAtial Sensing & Computing Lab

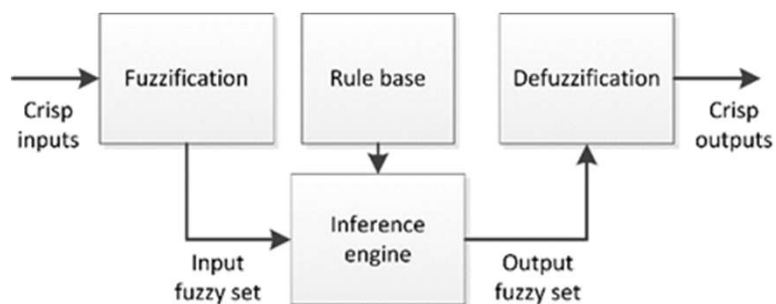


Key Concepts in Fuzzy Logic

1. **Fuzzy Sets:** A fuzzy set is a set without a sharp boundary. The membership of elements in a fuzzy set is described by a membership function that assigns each element a degree of membership ranging from 0 to 1.
2. **Membership Functions:** These functions define how each point in the input space is mapped to a membership value (or degree of membership) between 0 and 1. Common membership functions include triangular, trapezoidal, and Gaussian functions.
3. **Linguistic Variables:** These are variables whose values are words or sentences from natural language rather than numerical values. For example, temperature might be described as "hot," "warm," or "cold."
4. **Fuzzy Rules:** These are if-then statements that form the basis of decision-making in fuzzy systems. For example, "If temperature is high, then fan speed is high."
5. **Fuzzy Inference System (FIS):** This system uses fuzzy set theory to map inputs to outputs. It involves fuzzification, rule evaluation, and defuzzification.



Fuzzy Logic Systems Architecture





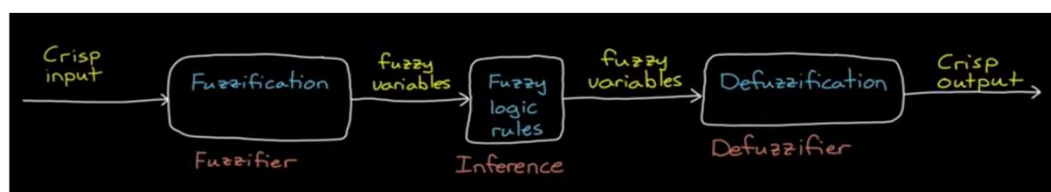
Fuzzy Logic Systems Architecture

- Has four main parts
 1. **Fuzzification Module:** transforms the system inputs, which are crisp numbers, into fuzzy sets. It splits the signal into 5 sets
 - 1. Large positive (LP) 2. Medium positive (MP) 3. small (S), 4. Median Negative (MN), 5. Large Negative (LN)
 2. **Knowledge Base (Rule base):** It stores IF-THEN rules provided by experts.
 3. **Inference Engine:** It simulates the human reasoning process by making fuzzy inference on the inputs and IF-THEN rules.
 4. **Defuzzification Module:** It transforms the fuzzy set obtained by the inference engine into a crisp value.



Algorithm

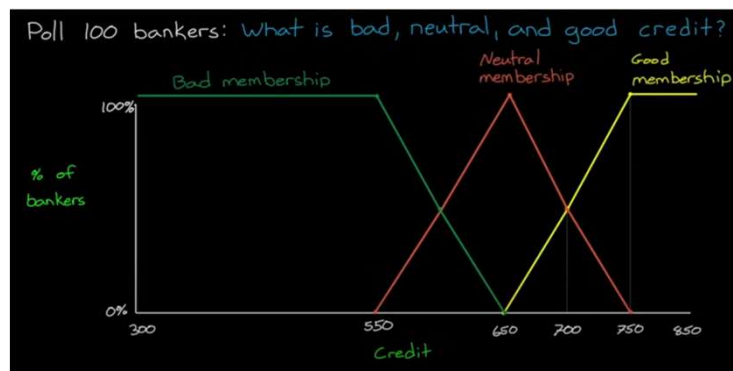
- Algorithm
 1. Define linguistic Variables and terms (start)
 2. Construct membership functions for them. (start)
 3. Construct knowledge base of rules (start)
 4. Convert crisp data into fuzzy data sets using membership functions (fuzzification)
 5. Evaluate rules in the rule base and combine results (inference engine)
 6. Convert output data into non-fuzzy values. (defuzzification)





Example –Bank loan risk based on credit score

- 1. Defining linguistic variables and terms
 - Credit= {Bad, Neutral, good}
 - Risk = {low, medium, high}
- 2. Construct membership Function



Example –Bank loan risk based on credit score

- 3. Construct knowledge base rule

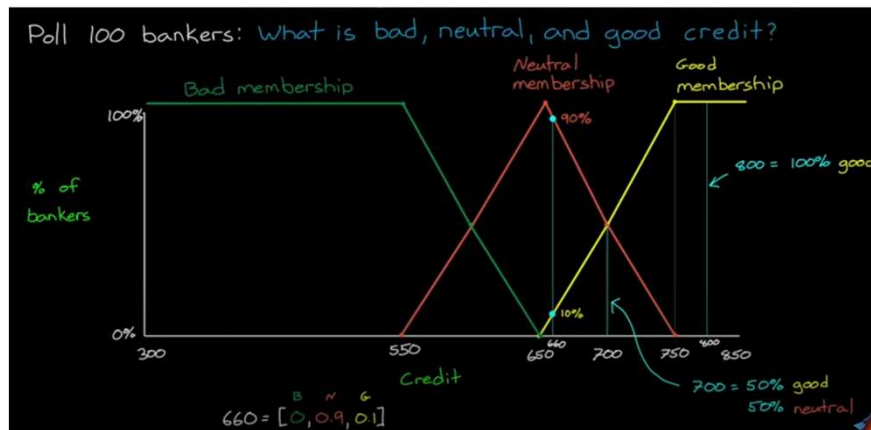
Rules

If good credit then low risk
If neutral credit then medium risk
If bad credit then high risk



Example –Bank loan risk based on credit score

- 4. Convert crisp data to fuzzy set (**Fuzzification**)
 - Example credit of 660 from the graph is [0% , 0.9 neutral, 0.1 good]



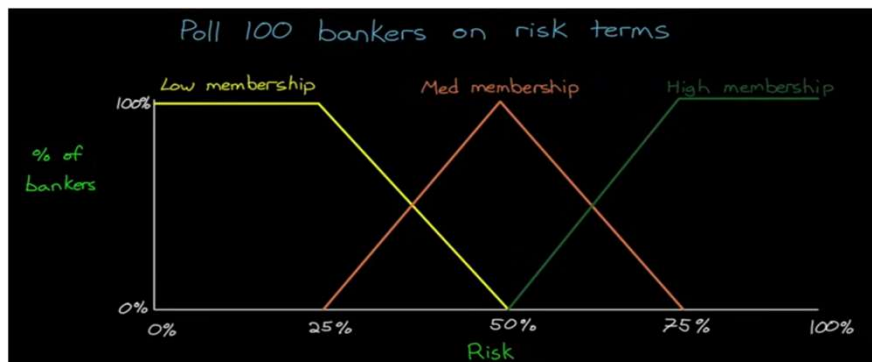
Example –Bank loan risk based on credit score

- 5. Evaluate rules in the rule based and combine the result (**Inference Engine**)
 - Based on the rules fuzzy output is also [0 bad, 0.9 neutral, 0.1good]
 - This is because we used simple rule, more complex rule can give different numbers



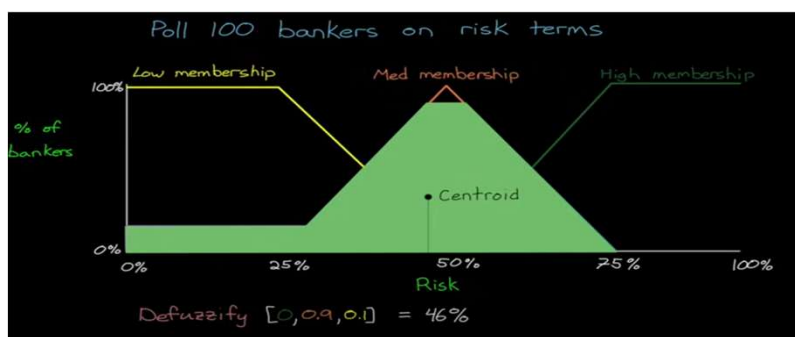
Example –Bank loan risk based on credit score

- 6. Convert output data into non-fuzzy values (**defuzzification**)
 - Get membership function for converting fuzzy risk terms (low, medium, high) into crisp % value



Example –Bank loan risk based on credit score

- 6. Convert output data into non-fuzzy values (**defuzzification**)
 - Use membership function for converting fuzzy risk terms (low, medium, high) into crisp % value



- The output value is the center of mass (or balance point) of the fuzzy region under the membership function curve.
- Therefore, the credit score of 660 is 46% risky



Application of Fuzzy Logic system

- **Control Systems:** Used in automation (e.g., washing machines, air conditioners) for adaptive decision-making.
- **Natural Language Processing (NLP):** Handles ambiguity in human language for better text and speech processing.
- **Expert Systems:** Mimics human reasoning for decision-making in healthcare, finance, and diagnostics.
- **Robotics:** Enables robots to navigate uncertain environments and perform complex tasks.
- **Image Processing:** Enhances pattern recognition and edge detection in computer vision.
- **Decision Support Systems:** Provides flexible solutions for uncertain or incomplete data.
- **Automotive Systems:** Improves anti-lock braking systems (ABS) and cruise control.



Natural Language Processing



Introduction

- **What is NLP?**
 - A subfield of AI that enables machines to understand, interpret, and generate human language.
 - Combines linguistics, computer science, and machine learning.
- **Why is NLP Important?**
 - Enables machines to interact with humans naturally (e.g., chatbots, virtual assistants).
 - Helps in automated translation (Google Translate).
 - Enhances information retrieval (search engines).
 - Supports sentiment analysis in social media.



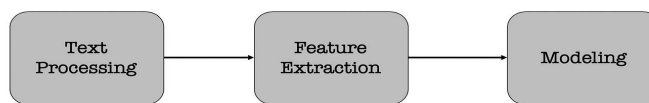
Key Concepts in NLP

- **Tokenization:**
 - Breaking text into words, sentences, or sub-words.
 - Example: "I love AI!" → ["I", "love", "AI", "!"].
- **Stemming and Lemmatization:**
 - Stemming is reducing words to their base forms by removing the suffixes (e.g., "running" → "run").
 - Lemmatization reduce words to their base while considering the context (e.g., "ran" → "run").
- **Part-of-Speech (POS) Tagging:**
 - Identifying grammatical roles (e.g., noun, verb, adjective).
 - Example: "She runs fast" → ["She" (pronoun), "runs" (verb), "fast" (adverb)].
- **Named Entity Recognition (NER):**
 - Detecting names, dates, locations, etc.
 - Example: "John works in New York" → ["John" (person), "New York" (location)].
- **Syntax and Semantics:**
 - Syntax: Understanding sentence structure (e.g., subject-verb-object).
 - Semantics: Understanding meaning (e.g., "bank" as a financial institution).



NLP Pipeline

- The common NLP pipeline consists of three stages:
 - Text preprocessing
 - Feature extraction
 - Modelling



Text Preprocessing Techniques

- **Tokenization:** Splitting text into words or sentences.
- **Stopword Removal:** Removing common words (e.g., "the", "and").
- **Stemming:** Reducing words to their root form (e.g., "running" → "run").
- **Lemmatization:** Converting words to their base form using linguistic rules.



Feature Extraction

1. **Bag of Words (BoW)** – Represents text as word frequency vectors.
2. **TF-IDF (Term Frequency - Inverse Document Frequency)** – improved version of BoW. Measures word importance in a document.
 - TF measures how often word appear in document
 - $TF = \frac{\text{Number of times word appear}}{\text{total words in the document}}$
 - IDF is how unique a word is across all documents
 - $IDF = \frac{\text{Total Number of documents}}{\text{number of documents containing the word}}$
 - TF-IDF score = $TF \times IDF$
 - Higher values mean the word is important for that document.
 - Emphasizes unique words that distinguish a document from others.
3. **Word Embeddings** – use vector(100 to 300 dimensions) to represent word. Examples of word embeddings approaches are Word2Vec, GloVe, BERT.



Feature Extraction

Feature	TF-IDF	Word Embeddings
Meaning	✗ (Ignores meaning)	✓ (Understands meaning)
Context Sensitivity	✗ (Treats words independently)	✓ (Considers context)
Dimensionality	High	Low (Efficient)
Use in Deep Learning	✗ (Mostly for ML models)	✓ (Used in deep NLP models)



NLP Models

- **Rule-Based Systems:**
 - Early NLP systems using handcrafted rules (e.g., grammar rules).
 - Example: Fuzzy logic rules for sentiment analysis.
- **Machine Learning:**
 - Algorithms like Naive Bayes, SVM, and decision trees.
 - Requires labeled data for training.
- **Deep Learning:**
 - Models like RNNs, LSTMs, and Transformers (BERT, GPT (generative pre-trained transformer)).
 - Handles sequential data and captures context.



Applications of NLP

1. **Speech Recognition** – Converts speech into text (Google Voice, Siri).
2. **Machine Translation** – Converts one language to another (Google Translate).
3. **Sentiment Analysis** – Determines emotions in text (social media monitoring).
4. **Chatbots & Virtual Assistants** – Conversational AI (ChatGPT, Alexa).
5. **Text Summarization** – Extractive and abstractive summarization.
6. **Text Generation** – AI-generated content, creative writing, and code generation (GPT models).



Challenges

- **Ambiguity:**
 - Words with multiple meanings (e.g., "bank", riverside or financial institution?).
 - Requires context to resolve.
- **Context Understanding:**
 - Sarcasm, idioms, and cultural nuances.
 - Example: "Great, I lost my phone again!" (sarcasm).
- **Data Quality:**
 - Noise, bias, and lack of labeled data.
 - Example: Biased training data leading to unfair predictions.
- **Computational Complexity:**
 - Large models like ChatGPT require significant resources.



Future Trends in NLP

- **Multilingual Models:**
 - Supporting low-resource languages (e.g., African or indigenous languages).
- **Explainable AI:**
 - Making NLP models more transparent and interpretable.
- **Conversational AI:**
 - More human-like chatbots with better context understanding and emotionally intelligent interaction.
- **Ethical AI:**
 - Addressing bias, fairness, and privacy in NLP systems.



Neural Networks



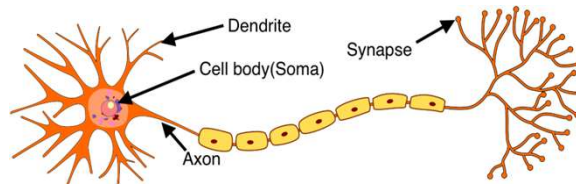
Introduction

- A neural network (ANN/NN) is a type of machine learning algorithm inspired by the human brain designed to recognize patterns and learn from data.
- It consists of interconnected artificial neurons that process inputs and produce outputs.
- Neural networks power **a wide range of applications** across various fields, including autonomous driving, robotics, medicine, Engineering, Finance, etc..



Biological neuron

- A **biological neuron** is a nerve cell that transmits electrical and chemical signals in the nervous system.
- It consists of **specialized structures** that help in receiving, processing, and transmitting information

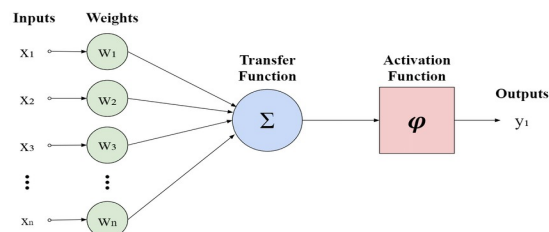


- Components of a Neuron:
 1. **Dendrites** which receives signals from other neurons
 2. **Cell Body (Soma)** processes the signals into output signal
 3. **Axon** which transmits the output signals from soma
 4. **Axon Terminals** sends signals to next neuron
 5. **Synapses** is the gap between two neurons where communication occurs



Artificial Neuron

- An **artificial neuron** is a **mathematical model** inspired by a biological neuron.
- It takes multiple inputs, processes them using weights and activation functions, and produces an output.

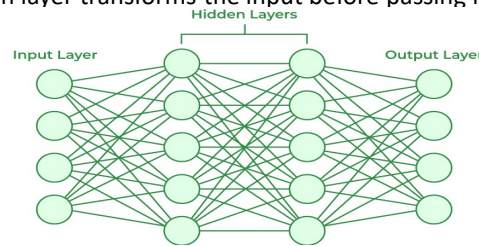


- Components
 1. **Inputs ($x_1, x_2, x_3, \dots$)** is the data which can be pixel value or numerical
 2. **Weights ($w_1, w_2, w_3, \dots$)** assigned to each input to represent its importance
 3. **Transfer Function** computes the weighted sum of the inputs
 4. **Activation Function** determines the output of a neuron
 5. **Output** is the final processed result



Neural Networks

- A **Neural Network** is a collection of interconnected **artificial neurons** that work together to process data and make predictions.
- It is organized into **layers**, where each layer transforms the input before passing it forward.



- Components of a Neural Network
 1. **Input Layer** which receives the raw data
 2. **Hidden Layer** Any layer between the input and the hidden layer
 3. **Output Layer** Produces the final result, such as classification labels.
- **Deep Learning** is a field that focus on using ANN with multiple hidden layers.



Training ANN

- **Input data:** First input data is prepared each with its target output.
- **Forward propagation:**
 - Inputs are multiplied by weights, summed, and activated across all layers until the output layer.
 - Outputs are compared to the actual target values and a loss error is computed
- **Backward propagation:**
 - The loss is propagated backward through the network.
 - Gradients of the loss with respect to each weight are calculated using the chain rule of calculus.
- **Weight update:** And the weights are updated based on the gradients.
- **Iteration:** The process of forward propagation, backward propagation, and weight update are repeated several times, until the network performs well.



Popular Types of Artificial Neural Networks

1. **Feedforward neural network (FFN):** In this network, information flow in one direction without loops.
2. **Convolutional Neural Networks (CNNs):** Uses convolutional layers to process grid-like data such as image.
3. **Recurrent Neural Networks (RNNs):** this network is designed to process sequential data with loops to retain memory.
4. **Generative Adversarial Networks (GANs):** are two neural networks (a Generator and a Discriminator) competing against each other to create and evaluate realistic data.
5. **Transformers:** This networks use self-attention to process and generate sequences.



Applications of NN

1. **Image Processing & Vision:** Image classification, object detection, segmentation. Uses CNN and Transformers.
2. **Natural Language Processing (NLP):** Text/sentiment analysis, machine translation, chatbots, speech recognition. Uses RNN and Transformers.
3. **Finance & Business:** Fraud detection, stock market prediction, recommender systems. Uses FNN, RNN and Transformers.
4. **Autonomous Systems:** Self-driving cars, robotics, real-time decision-making. Uses CNN, RNN and Transformers.
5. **Creative Applications:** Art generation, AI music, game AI. Uses GAN, RNN and Transformers.
6. **Language Models:** Text generation, question answering, conversational AI. Uses Transformers.